SDI 4213-980: DevOps – CI/CD
Week 11 Individual Exercise

Deploy the Help Desk Application to the Cloud

Individual assignment: Complete the work independently using your own GitHub repository and cloud service. Screenshots are required as evidence of each milestone.

Purpose

In this exercise you will take the containerized Help Desk application from local execution to a real cloud deployment. You will create a managed PostgreSQL database, deploy the Dockerized application as a public web service, configure runtime settings securely, verify health and persistence, and demonstrate a version update through a second deployment.

Learning Outcomes

- Deploy a Dockerized application to a managed cloud platform.
- Connect an application to managed PostgreSQL using an environment-supplied connection string.
- Configure cloud environment variables without committing secrets.
- Verify a public HTTPS endpoint and application health.
- Use deployment logs to confirm or troubleshoot cloud deployment.
- Demonstrate that persistent data survives application redeployment.
- Connect Git/GitHub, CI, Docker, and cloud deployment into one workflow.

Required Accounts and Software

- GitHub account and Git installed.
- Docker Desktop installed and running.
- Python 3.13 or a compatible course Python installation.
- A free Render account for the cloud deployment.
- VS Code or another code editor.

Cost requirement: Use only the Free plan for this exercise. Do not upgrade to a paid plan and do not enter payment information for the assignment. Render currently documents a no-payment-required first deployment path and free web/database resources for learning and preview use.

Starter Repository

- A working Streamlit Help Desk / Service Request Tracker.
- SQLite fallback for local development.
- Managed PostgreSQL support through DATABASE_URL.
- A Dockerfile prepared to use the cloud provider PORT value and 0.0.0.0.
- A Docker health check using Streamlit /_stcore/health.
- GitHub Actions CI that runs tests and builds the Docker image.

- .env.example, .gitignore, and .dockerignore.
- Screenshot checklist and troubleshooting notes.

Important Cloud Free-Tier Notes

- A free Render web service may spin down after inactivity. The first request after inactivity can therefore be slower while the service spins back up.
- Free web services do not provide persistent local disks. This is why your ticket records must live in managed PostgreSQL, not inside the web container.
- Free Render PostgreSQL databases currently expire after 30 days. That is sufficient for this course exercise, but it is not a production design.
- Never include database passwords, database URLs, API keys, or tokens in your screenshots.

Git Workflow Requirement

```
Issue / Task
  ↓
Branch
  ↓
Change
  ↓
Commit
  ↓
Push
  ↓
Pull Request
  ↓
CI
  ↓
Merge to main
  ↓
Cloud Deployment
```

Part A – Verify the Starter Locally

1. Extract the starter package and place it in your Week 11 working folder.
2. Create a new GitHub repository for the Week 11 exercise and push the starter files.
3. Create and activate a Python virtual environment.
4. Install dependencies with: python -m pip install -r requirements.txt
5. Run the automated tests with: python -m pytest -v
6. Run the application locally with: python -m streamlit run app.py
7. Create one local service request so you understand the application before deploying it.

SCREENSHOT MILESTONE 1 **Show the terminal with all automated tests passing AND enough of the project path to demonstrate you are in the Week 11 repository.**

Part B – Verify the Docker Image Locally

```
docker build -t helpdesk-cloud:week11 .
docker run --rm -p 8501:8501 -e DEPLOY_ENV=Local-Docker helpdesk-cloud:week11
```

1. Build the supplied Dockerfile.
2. Run the image locally and open http://localhost:8501.
3. Confirm the page reports Environment: Local-Docker.
4. Stop the container when finished.

SCREENSHOT MILESTONE 2
Show the Help Desk application running from the Docker container in your browser. The deployment caption should show Local-Docker.

Part C – Confirm GitHub CI

1. Push the starter repository to GitHub.
2. Open the Actions tab.
3. Confirm the CI workflow runs tests and successfully builds the Docker image.
4. If CI fails, use the workflow logs to diagnose the problem before continuing.

SCREENSHOT MILESTONE 3
Show the GitHub Actions workflow with the test-and-build job passing.

Part D – Create Managed PostgreSQL

1. Sign in to Render.
2. Create a new PostgreSQL database using the Free plan.
3. Choose a region and record that region for the next step.
4. Give the database a recognizable name such as week11-helpdesk-db.
5. After creation, locate the Internal Database URL. Treat this URL as a secret because it contains connection credentials.
6. Do not paste the database URL into your source code or commit it to GitHub.

SCREENSHOT MILESTONE 4
Show the managed PostgreSQL resource in Available/Running status, including the database name and region. DO NOT show the Internal Database URL, password, or credentials.

Part E – Create the Cloud Web Service

1. In Render, create a new Web Service.

2. Connect the GitHub repository containing your Week 11 starter application.
3. Select Docker as the runtime/language so Render uses the supplied Dockerfile.
4. Choose the same region used for the PostgreSQL database.
5. Select the Free compute plan.
6. Leave automatic deployment from the main branch enabled unless your instructor directs otherwise.

SCREENSHOT MILESTONE 5
Show the cloud Web Service configuration or service overview demonstrating that the service is connected to your GitHub repository, uses Docker, and is on the Free plan. Do not expose credentials.

Part F – Configure Runtime Environment Variables

Variable	Value
DEPLOY_ENV	Cloud
APP_VERSION	1.0.0
DATABASE_URL	The Internal Database URL from your managed PostgreSQL resource

1. Open the web service environment-variable settings.
2. Add DEPLOY_ENV and APP_VERSION.
3. Add DATABASE_URL using the managed PostgreSQL Internal Database URL.
4. Do not add PORT manually unless the platform documentation specifically requires it; the platform supplies it at runtime.
5. Set the health check path to /_stcore/health if the service configuration provides a health-check field.

SCREENSHOT MILESTONE 6
Show the environment-variable NAMES configured in the cloud dashboard. Secret values must be hidden or masked. The screenshot must not reveal DATABASE_URL.

Part G – Deploy and Read the Logs

1. Start or trigger the deployment.
2. Open the deploy logs while the service builds.
3. Identify the stages where dependencies are installed, the Docker image is built, and the Streamlit service starts.
4. Confirm the deployment reaches a successful/live status.
5. If it fails, diagnose from the logs rather than randomly changing settings.

SCREENSHOT MILESTONE 7
Show the successful deployment status AND a portion of the deployment/runtime logs demonstrating that the application started successfully.

Part H – Verify the Public HTTPS Application

1. Open the public HTTPS URL assigned to your web service.
2. Confirm the application displays Environment: Cloud.
3. Confirm Database: PostgreSQL is displayed.
4. Create a cloud service request with a distinctive title such as Week 11 Cloud Test - <your initials>.
5. Change its status to In Progress or Closed.

SCREENSHOT MILESTONE 8
Show the public HTTPS Help Desk application with Environment: Cloud, Database: PostgreSQL, and your newly created cloud ticket visible. You may crop the browser to avoid unrelated personal information.

Part I – Prove Persistence

1. Use the cloud dashboard to redeploy the current service or restart the web service.
2. After the new application instance is running, return to the public URL.
3. Verify the ticket created in Part H is still present.
4. Explain in one or two sentences why the data survived even though the application instance was replaced/restarted.

SCREENSHOT MILESTONE 9
Show the same cloud ticket still present after the redeploy/restart. Include enough of the page to show Cloud + PostgreSQL so the persistence evidence is clear.

Part J – Deploy Version 1.1.0

1. Create a new Git branch named feature/week11-version-update.
2. Change APP_VERSION in the cloud environment from 1.0.0 to 1.1.0 OR make a small visible application improvement approved by your instructor.
3. Make a small README update describing your cloud URL and deployment process. Do not place credentials in README.
4. Commit and push your change.
5. Open a pull request and confirm CI passes.
6. Merge to main.
7. Confirm the cloud platform deploys the updated main branch.
8. Verify the public application shows Version 1.1.0 and still contains your PostgreSQL ticket data.

SCREENSHOT MILESTONE 10
Show (a) the merged GitHub pull request or passing final CI run and (b) the public cloud application displaying

Version 1.1.0 with the existing ticket still visible. Two images may be used for this milestone.

Required Submission

- GitHub repository URL.
- Public cloud application URL.
- All 10 screenshot milestones in order and clearly labeled.
- Short reflection responses.
- Evidence that the final GitHub CI workflow is passing.

Screenshot format: Place screenshots into a single Word or PDF submission in milestone order. Each image should have a caption such as "Milestone 7 - Successful cloud deployment." Screenshots must be readable and must not expose passwords, database URLs, tokens, or other secrets.

Grading

Category	Points
Milestones 1-3: local, Docker, and CI verification	20
Milestones 4-7: cloud database, web service, configuration, deploy logs	30
Milestones 8-9: public application and persistence	25
Milestone 10: version update, PR/CI, final deployment	15
Total	100

Troubleshooting Checklist

Problem	Check
Cloud build fails	Read build logs; verify requirements.txt and Dockerfile are committed.
Application will not load	Confirm runtime logs, 0.0.0.0 binding, and PORT handling.
Database connection fails	Confirm DATABASE_URL is correct, hidden, and database is available in the expected region.
App says SQLite in cloud	DATABASE_URL is missing or not recognized.
Ticket disappears	Confirm the application was using PostgreSQL, not local SQLite/container storage.
New commit does not appear	Check branch, merge status, CI, and cloud deployment history.

Current Platform References

Render Free Resources: https://render.com/docs/free

Render Web Services: https://render.com/docs/web-services

Render First Deploy: https://render.com/docs/your-first-deploy

Streamlit Docker Deployment: https://docs.streamlit.io/deploy/tutorials/docker